

3. Encapsulation :-

→ composition.

Class Engine:-

```
def start(self):  
    print("Engine starting")
```

Class Car:

```
def __init__(self):  
    self.engine = Engine()  
  
def start(self):  
    print('start engine first')  
    self.engine.start()  
    print('Car has started')
```

→ Dynamic extension:-

Class Text:

```
def render(self):  
    return "Hello".
```

```
def render(self):  
    return f"<h> {self.wrapper.render()} </h>
```

4. Polymorphism and Duck typing.

↓
dynamic typing

dynamic adapts based on input provided.

Class variables (vs) Instance variables.

→ scope of a particular variable.

→

class Student:

school = "ABC School" → class variable.

def __init__(self, name)

self.name = name → instance variable

↓
Object specific variable.

↙
assigned to the
whole class.

Student.school() = "xyz School"

s1.school() =

Instance Methods :-

- Take self as first argument
- Can access both instance and class variables.
- ↓
- Can modify as well.

Class Method :-

- cls as input parameter.
- Can access only class variable.

@ Class method :

Can't access instance
variables

```
def get_instance_count(cls):  
    return cls.rk.
```

Static Method :-

Decorators

- Modify the existing functions functionality to behave in a certain way
- modify and enhance without changing method.
- def

@ data class

```
class Person:  
    name: str  
    age: int
```

→ Key value pairs
instead of self

and strict data type checking.

pydantic uses data class

↓
byproduct of decorators.

def __init__(self):

→ How long is the class (SRP)

→ only one responsibility per class

Important class